

1. For the powers of 2, they occur at 1, 2, 4, 8, ..., $2^k \leq n$, each expensive, operation cost $\geq 2^k$. Let's assign each operation an amortized cost of 3.

(1) For non-power-of-2 operations, actual $\geq 1 \rightarrow$ save 2 credits.
 (2) For power-of-2 operations, we'll spend the credits we've saved since last power of 2.
 Between $2^{k-1} + 1$ and $2^k - 1$, there are $(2^{k-1}) - (2^{k-1} + 1) + 1 = 2^{k-1} - 1$ cheap operations $\rightarrow$ save $2 \times (2^{k-1} - 1) \geq 2^{k-1}$ credits, just enough to pay the next cost 2^k .
 $\therefore$ Amortized cost per operation ≤ 3 . Hence total actual cost $\leq 3n$.

2. We perform n operations where the i th operation has $C_i = f(i)$, if i is a power of 2, 1, 0 otherwise.

We use the potential method to compute the amortized cost.
 $\bar{C}_i = C_i + (\Phi(i) - \Phi(i-1))$
 Let $t(i) = 2^{\lfloor \log_2 i \rfloor}$ (the largest power of 2 $\leq i$) and define $\Phi(i) = i - t(i)$.

Define the potential function as: $\Phi(i) = 2 \cdot S(i) \geq 2 \cdot C_i - 2^{\lfloor \log_2 i \rfloor}$

(1) When i is not a power of 2.
 Actual cost $C_i = 1$, the potential increases by 2 since $\Phi(i) = 2 \cdot S(i-1) + 1 - 2 \cdot S(i-1) = 1$, therefore, $\bar{C}_i = 1 + 2 = 3$.
 (2) When $i = 2^k$.

Actual cost $= 2^k$. Right before this step there were $2^{k-1} - 1$ cheap operations since 2^{k-1} , so $\Phi(i-1) = 2 \cdot S(2^{k-1} - 1) = 2^k - 2$.

$\Phi(i) = 0$, Hence, $\bar{C}_i = 2^k + (0 - (2^k - 2)) = 2$.

Conclusion: Every operation has amortized cost ≤ 3 . Because $\Phi_n = 0$ and $\Phi_0 \geq 0$, $\sum_{i=1}^n \bar{C}_i = \sum_{i=1}^n C_i - (\Phi_n - \Phi_0) \leq \sum_{i=1}^n \bar{C}_i \leq 3n$.

Amortized cost per operation is $O(1)$, and total actual cost $\leq 3n$.

3. Yes. If we repeatedly insert n elements and perform only decrease-key operations (never extract-min), each decrease-key can cut one node and make it a child of another depending on how cascading cuts are postponed.

More concretely:
 1. Insert n elements with decreasing keys (so the heap keeps a chain).
 2. If we do consolidation (never extract-min), the heap may consist of a single tree with height n .

Thus the heap structure can degenerate into a linear chain — height $O(n)$ not $O(\log n)$.

4. $A^n = \begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix}$ $F_0 = 0, F_1 = 1, F_{k+1} = F_k + F_{k-1}$

We can prove that by mathematical induction.

When $n=1$, $A^1 = A = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} F_2 & F_1 \\ F_1 & F_0 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$

If $\exists n \geq 1$ s.t. $A^n = \begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix}$, then

$A^{n+1} = \begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} F_{n+1} + F_n & F_{n+1} \\ F_n + F_{n-1} & F_n \end{bmatrix} =$

$\begin{bmatrix} F_{n+2} & F_{n+1} \\ F_{n+1} & F_n \end{bmatrix}$ ($F_{n+2} = F_{n+1} + F_n, F_{n+1} = F_n + F_{n-1}$)

$\therefore \forall n \geq 1$, it's true that $A^n = \begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix}$

5. We can use Kruskal with pre-included edges

(1) Initialize the union-find (disjoint-set) structure and union all edges in A first. (If A already forms a cycle, no spanning tree can include A , but A is given to be acyclic.)
 (2) Then perform the usual Kruskal process on the remaining edges $E \setminus A$.

Sort edges by weight.
 For each edge $e \in E \setminus A$: if it connects 2 different components, add e to T and union its endpoints.

(3) Continue until $|T| = |V| - 1$

(4) Output T .

Correctness justification:

Feasibility: Because A is acyclic, it forms a forest that can always be extended to a spanning tree.

Optimality (Cut and Exchange Property):

Contractivity A turns each of its connected components into a vertex.

Running MST on the contracted graph G_A selects exactly the minimum-weight edge needed to connect those components.

If any other spanning tree T' containing A had smaller weight, the corresponding edge set in G_A would have smaller weight than the MST of G_A , contradicting optimality.

Hence, $T = A \cup B$ is the unique MST that contains A .

Time complexity

Same as the standard MST algorithm: $O(E \log V)$

Contraction and union-find operations are nearly linear.

6. (a) MST uniqueness

If all edge weights are distinct, the MST is unique. Proof: Every cut has a unique lightest edge, and that edge must be in the MST.

(b) Second-best MST Non-Unique

You can change one edge of the MST to the next-lightest edge in some cycle. Different cycles may produce the same total weight, so the second-best tree need not be unique.

(c) Shortest Path Uniqueness

Even with distinct edge weights, shortest paths between two vertices need not be unique. Example: Triangle $A-B-C$ with weight $A-B=1, B-C=2, A-C=3$.

Shortest $A \rightarrow C$ path is unique ($1+2=3$), but if we modify to $A-B=1, B-C=2, A-C=3$, then slightly adjust weights can create ties.

7. Construct a directed graph: $s \rightarrow a(i)$

$s \rightarrow b(i)$

$a(i) \rightarrow b(i)$

Shortest path $s \rightarrow b \geq 2$ via direct edge, but Dijkstra may relax $s \rightarrow a \rightarrow b$ first:

Extract s , relax edges (s,a) and (s,b)

Next extract a ($d[a]=1$), relax (a,b) giving $alt=3$, same as b 's dist.

So edges on the shortest path ab may be relaxed before those not of order.

Another example with different ordering: add more edges to path edges get processed non-sequentially by distance tie.

8. We define $S_k(x,y) = \min_{p: x \rightarrow y \text{ exp}} \max_{e \in p} w(e)$

Use a modified Dijkstra:

for each v : $dist[v] = \infty$

$dist[s] = 0$

Priority Queue $Q = \{ (s,s) \}$

While Q not empty:

$(d,u) = \text{extract-min}(Q)$

for each edge (u,v,w) :

$alt = \max(d,u,w)$

if $alt < dist[v]$:

$dist[v] = alt$

decrease-key (Q,v,alt)

This propagates the minimum possible bottleneck value to each v . Time $= O(m \log n)$ with binary heap.

Correctness: By monotonicity of max, Dijkstra-style proof applies.